

CODE DFS-

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
bool DFS_Search(int node, vector<vector<int>> &adj, vector<bool> &visited, int target)
```

```
{
```

```
    visited[node] = true;
```

```
    // Display current node
```

```
    cout << "Visiting node: " << node << endl;
```

```
    // Check if target found
```

```
    if (node == target)
```

```
        return true;
```

```
    // Visit all adjacent nodes
```

```
    for (int neighbor : adj[node])
```

```
    {
```

```
        if (!visited[neighbor])
```

```
        {
```

```
            if (DFS_Search(neighbor, adj, visited, target))
```

```
                return true;
```

```
        }
```

```
    }
```

```
    return false;
}

int main()
{
    int vertices, edges;

    cout << "Enter number of vertices: ";
    cin >> vertices;

    cout << "Enter number of edges: ";
    cin >> edges;

    vector<vector<int>> adj(vertices);

    cout << "Enter edges (source destination):\n";
    for (int i = 0; i < edges; i++)
    {
        int src, dest;
        cin >> src >> dest;
        adj[src].push_back(dest);
        adj[dest].push_back(src); // remove this line for directed graph
    }

    int start, target;
```

```

cout << "Enter starting node: ";
cin >> start;

cout << "Enter target node: ";
cin >> target;

vector<bool> visited(vertices, false);

bool found = DFS_Search(start, adj, visited, target);

if (found)
    cout << "Target " << target << " found in graph.\n";
else
    cout << "Target " << target << " NOT found in graph.\n";

return 0;
}

```

INPUT AND OUTPUT-

Enter number of vertices: 6

Enter number of edges: 5

Enter edges (source destination):

0 1

0 2

1 3

1 4

2 5

Enter starting node: 0

Enter target node: 4

Visiting node: 0

Visiting node: 1

Visiting node: 3

Visiting node: 4

Target 4 found in graph.

CODE BFS-

```
#include <iostream>
```

```
#include <vector>
```

```
#include <queue>
```

```
using namespace std;
```

```
bool BFS_Search(int start, vector<vector<int>> &adj, int target, int vertices)
```

```
{
```

```
    vector<bool> visited(vertices, false);
```

```
    queue<int> q;
```

```
    visited[start] = true;
```

```
    q.push(start);
```

```
    while (!q.empty())
```

```

{
    int node = q.front();
    q.pop();

    // Display current node
    cout << "Visiting node: " << node << endl;

    // Check if target found
    if (node == target)
        return true;

    // Visit neighbors
    for (int neighbor : adj[node])
    {
        if (!visited[neighbor])
        {
            visited[neighbor] = true;
            q.push(neighbor);
        }
    }
}

return false;
}

```

```

int main()

```

```
{  
    int vertices, edges;  
  
    cout << "Enter number of vertices: ";  
    cin >> vertices;  
  
    cout << "Enter number of edges: ";  
    cin >> edges;  
  
    vector<vector<int>> adj(vertices);  
  
    cout << "Enter edges (source destination):\n";  
    for (int i = 0; i < edges; i++)  
    {  
        int src, dest;  
        cin >> src >> dest;  
        adj[src].push_back(dest);  
        adj[dest].push_back(src); // remove for directed graph  
    }  
  
    int start, target;  
  
    cout << "Enter starting node: ";  
    cin >> start;  
  
    cout << "Enter target node: ";
```

```
cin >> target;

bool found = BFS_Search(start, adj, target, vertices);

if (found)
    cout << "Target " << target << " found in graph.\n";
else
    cout << "Target " << target << " NOT found in graph.\n";

return 0;
}
```

INPUT AND OUTPUT-

Enter number of vertices: 6

Enter number of edges: 5

Enter edges (source destination):

0 1

0 2

1 3

1 4

2 5

Enter starting node: 0

Enter target node: 4

Visiting node: 0

Visiting node: 1

Visiting node: 2

Visiting node: 3

Visiting node: 4

Target 4 found in graph.